

PHYS 2550 - Applied Machine Learning in Physics

Homework 1: Linear Regression from Scratch

Due: September 19, 2026

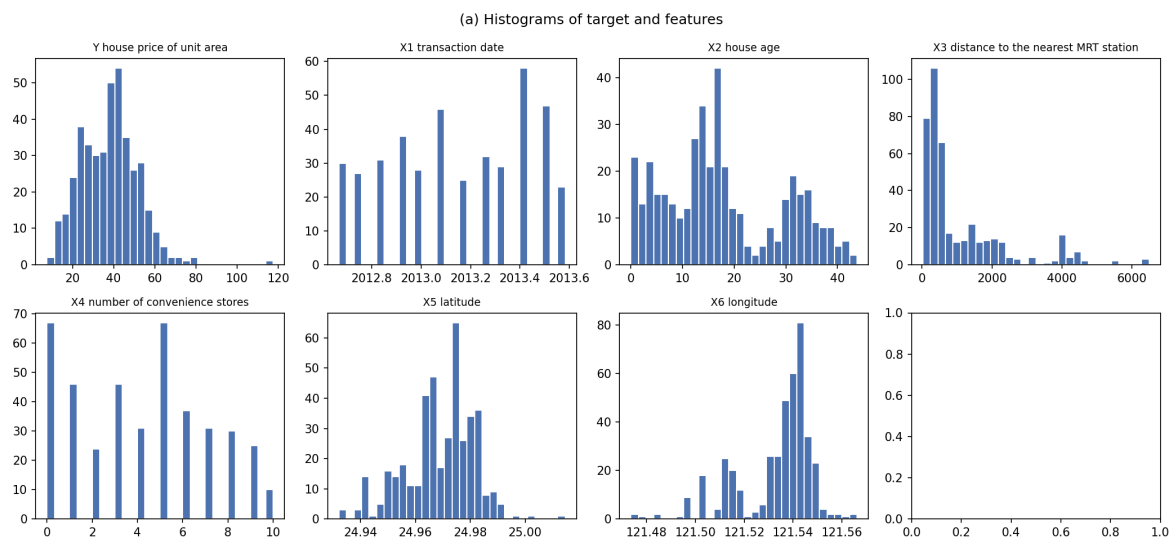
All model fitting below is hand-written NumPy (my own gradient-descent loops); no machine-learning libraries were used. Code: `hw1_analysis.py`. [notebook: [Colab/GitHub link](#)]

(a) The data

The data set ships as an Excel file, so I converted it once with pandas (`read_excel` → `to_csv`) and worked with `real_estate.csv` from then on. It contains **414 rows** (Taipei house transactions) and, to my relief, **no missing values**. Table 1 summarises the columns; Figure 1 shows a histogram of the target and each feature.

Table 1: Columns of the real estate data set, with units and observed ranges.

Column	Meaning (units)	Range
X1 transaction date	transaction date (year + fraction)	2012.667 – 2013.583
X2 house age	house age (years)	0 – 43.8, mean 17.71
X3 distance to the nearest MRT station	distance to nearest MRT station (metres)	23.4 – 6488.0, mean 1083.9
X4 number of convenience stores	convenience stores nearby (count)	0 – 10
X5 latitude	latitude (degrees)	24.932 – 25.015
X6 longitude	longitude (degrees)	121.474 – 121.566
Y house price of unit area	price per unit area (10,000 TWD / ping)	7.6 – 117.5, mean 37.98

**Figure 1:** Histograms of the target Y and the six features.

Things that struck me as odd or worth remembering:

- No missing values at all – a pleasant surprise for a real-world data set.
- X3 is strongly right-skewed: a few houses sit more than 4000 m from any MRT station, one as far as 6488 m.

- One house costs 117.5 (10k TWD/ping), far above the pack – a clear outlier.
- X3's scale (thousands of metres) dwarfs X2's (0–44 years). This comes back to bite naive gradient descent in part (c).
- X1 lives on a discrete monthly grid and X4 is integer counts; latitude and longitude fall in tight clusters.

(b) One feature: price vs. house age

As a warm-up I fit a one-feature linear model on house age (X2),

$$\hat{y} = w_0 + w_1 x$$

with the mean-squared-error loss

$$L(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (w_0 + w_1 x_i - y_i)^2$$

For gradient descent I need the two partial derivatives. Differentiating the sum term by term and applying the chain rule (the outer square gives a factor 2, the inner term gives 1 for w_0 and x_i for w_1):

$$\begin{aligned} \frac{\partial L}{\partial w_0} &= \frac{1}{N} \sum_{i=1}^N 2(w_0 + w_1 x_i - y_i) \cdot 1 = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \\ \frac{\partial L}{\partial w_1} &= \frac{1}{N} \sum_{i=1}^N 2(w_0 + w_1 x_i - y_i) \cdot x_i = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) x_i \end{aligned}$$

Picking the learning rate was trial and error: my first guess, $\text{lr} = 0.01$, diverged (the loss exploded), so I kept halving it until the run stayed stable, ending at **lr = 0.001**. I then ran **50,000 epochs**, stopping once the loss curve had visibly flattened (Figure 2). The final parameters are **$w_0 = 42.4347$** , **$w_1 = -0.2515$** , with final **MSE = 176.5005**.

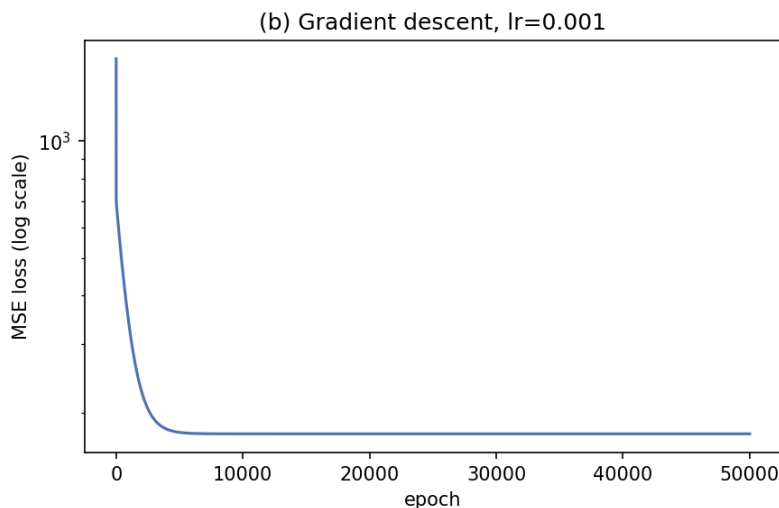


Figure 2: Gradient-descent loss vs. epoch for the one-feature model (log scale on y).

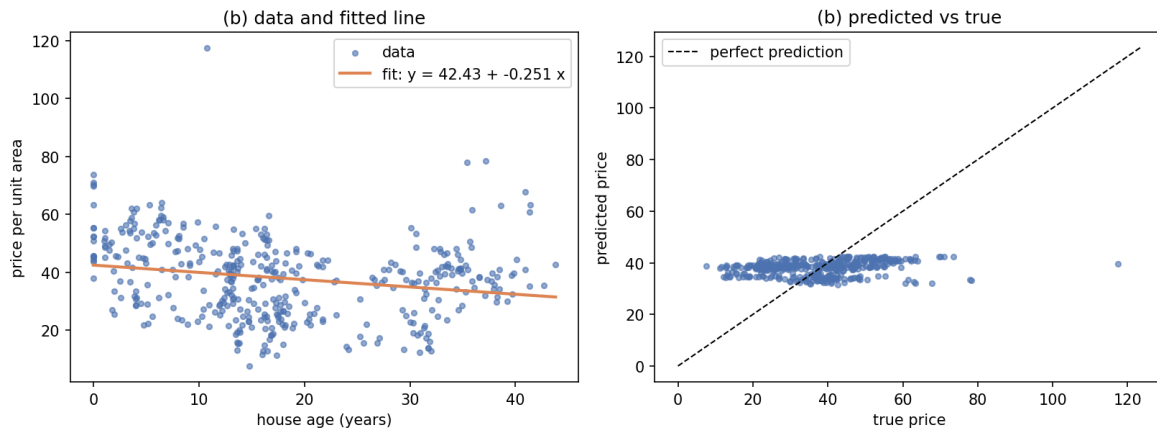


Figure 3: Left: house-age data with the fitted line. Right: predicted vs. true price.

Verdict: house age on its own is a **weak predictor**. The slope is small and negative (older houses are slightly cheaper), but the scatter around the line is huge (Figure 3, left). Consequently the predictions are compressed into roughly 30–45 while the true prices span 7.6–117.5 (Figure 3, right) – the model explains only a sliver of the variance.

(c) More features, and why normalisation matters

Next I used three features together – house age (X2), distance to the nearest MRT station (X3) and number of convenience stores (X4) – in the vector model

$$\hat{y} = \mathbf{w}^T \tilde{\mathbf{x}}, \quad \tilde{\mathbf{x}} = (1, x_2, x_3, x_4)^T$$

where the constant 1 is appended so the intercept lives inside $\mathbf{w}$. Stacking all 414 rows into the design matrix $\mathbf{X}$, the loss and its gradient become

$$L(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2, \quad \nabla_{\mathbf{w}} L = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Running this on the **raw** features went badly. With $\text{lr} = 10^{-6}$ the loss diverged (it passed 10^{15} by epoch 10); with $\text{lr} = 10^{-7}$ it was stable but crawled – the MSE was still 996.36 after 5000 epochs (Figure 4). After **normalising** each feature (subtract the mean, divide by the standard deviation) I could use $\text{lr} = 0.01$, and 5000 epochs gave $\mathbf{w} = [37.9802, -2.8772, -6.7808, 3.8171]$ (intercept, age, MRT distance, stores) with **MSE = 84.7607** – less than half of part (b)'s 176.50.

The reason is geometry. Distance spans 0–6500 m while age spans 0–44 years, so the curvature of the loss bowl along the distance direction is roughly $(6500/40)^2 \approx 10^4$ – 10^6 times larger than along the age direction. A single learning rate cannot serve both: large enough for age diverges on distance, safe for distance barely moves at all. Normalising equalises the curvatures, so one learning rate works for every direction at once.

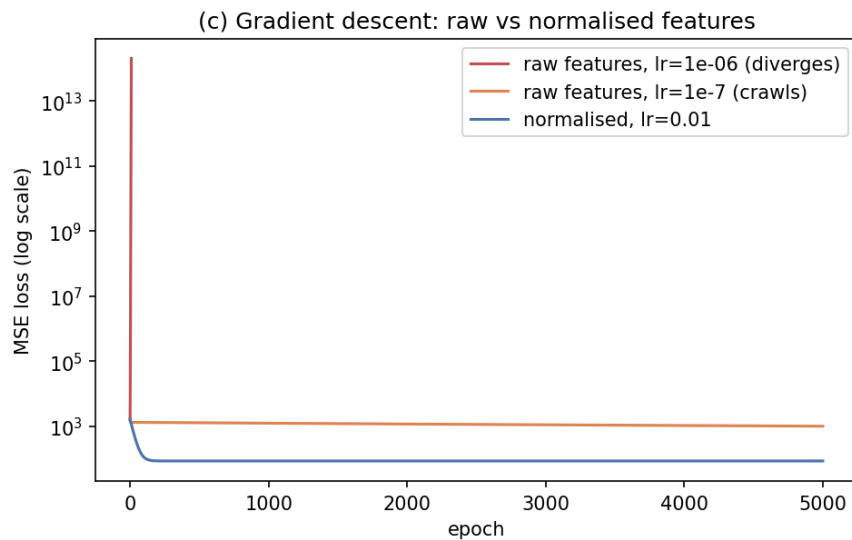


Figure 4: Loss curves on raw vs. normalised features. Raw $lr = 1e-6$ diverges; raw $lr = 1e-7$ is stable but barely descends; normalised features converge fast.

(d) What did we learn?

Final losses: **176.50** for the one-feature model of part (b) vs. **84.76** for the three-feature normalised model of part (c) – adding features and fixing the scaling cut the error by more than half. Figure 5 compares both models' predictions against the true prices on the same axes, and Figure 6 plots price against each feature with the correlation coefficients annotated.

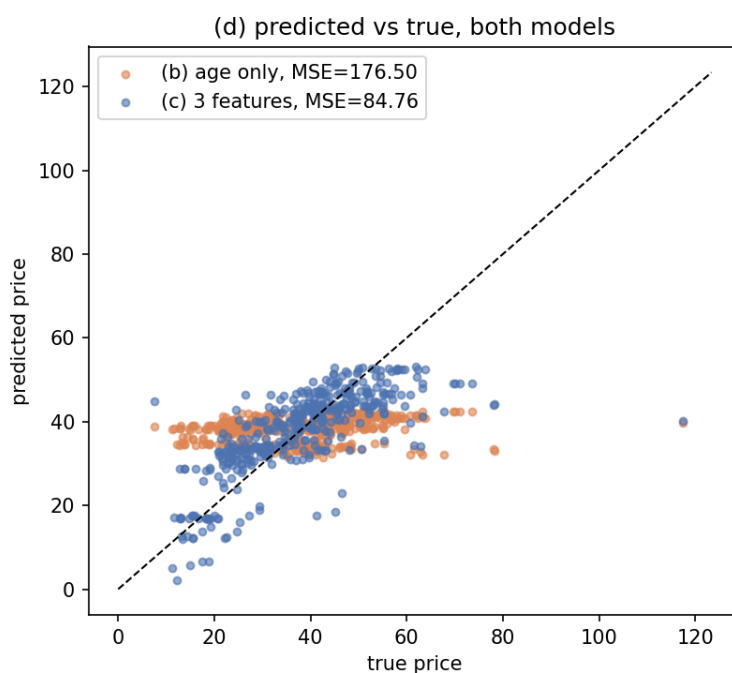


Figure 5: Predicted vs. true price for the models of (b) and (c) on the same axes.

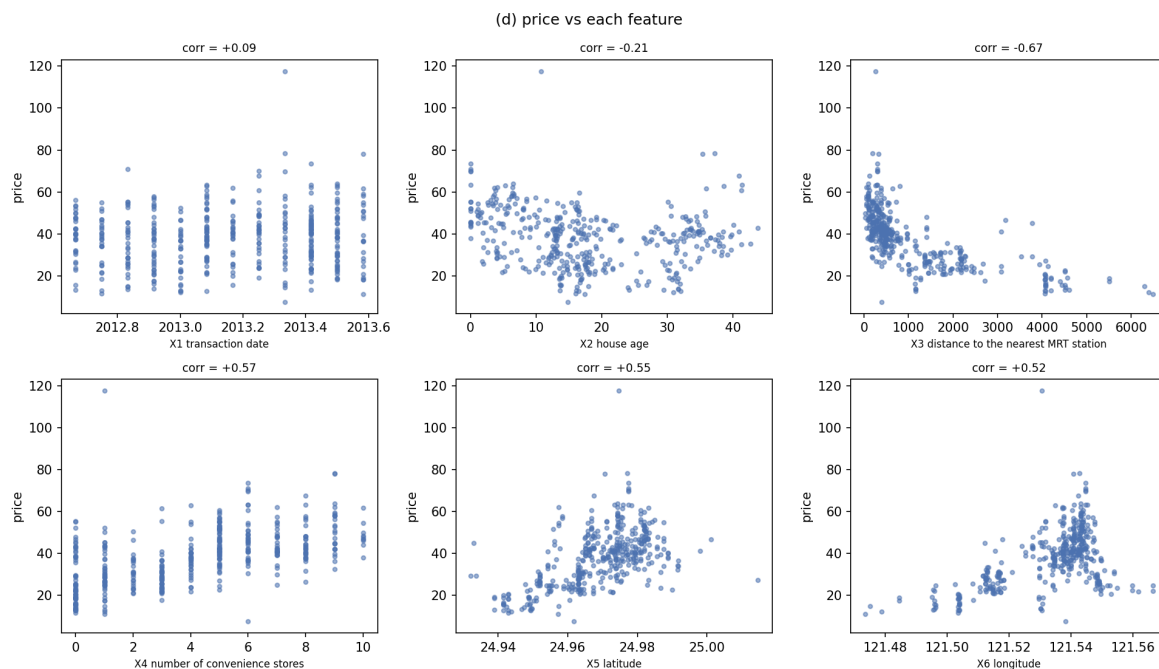


Figure 6: Price per unit area vs. each feature, with Pearson correlations.

The correlations with price are: X3 -0.67, X4 +0.57, X5 +0.55, X6 +0.52, X2 -0.21, X1 +0.09.

Which features are informative? Distance to the MRT station is the strongest single signal, followed by the number of convenience stores and the location (latitude/longitude). House age is weak, and the transaction date carries essentially nothing.

What limited the model? Four things: the outliers (the 117.5 house and the far-from-MRT houses pull the fit around); the feature scales, which forced me to normalise before gradient descent behaved; the visible saturation in price-vs-distance, which a straight line cannot capture; and the fact that location enters only linearly, although its effect is almost certainly not a plane.

What would I change in a second version? Add latitude and longitude to the feature set, log-transform X3 to tame its skew, treat the outliers explicitly, and use a proper train/test split so the reported error means something.

Bonus: closed-form check with the normal equation

As a sanity check I solved the same normalised three-feature problem in closed form with the normal equation

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$$

computed with `np.linalg.solve` rather than an explicit inverse. The result is $\mathbf{w}^* = [37.980193, -2.877175, -6.780847, 3.817079]$ with $\text{MSE} = 84.760706$. My gradient descent matches it to $\max |\Delta \mathbf{w}| = 1.71 \times 10^{-13}$ after 5000 epochs, and it gets within 0.01% of the closed-form loss after only 336 epochs.

My working definition of “close enough”: a relative loss gap below 10^{-4} . Past that point the remaining epochs only polish decimals of the weights that no downstream prediction will ever notice.